Common setup (run once)

```
pip install nltk
```

```
import nltk

# Download the NLTK datasets/models needed by the lab programs.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('brown')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('universal_tagset')
nltk.download('conll2000')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

Run each program in Jupyter Notebook, Google Colab, or VS Code with Python installed. Some programs download corpora on first run and need an internet connection.

7. Calculate bigrams from a given corpus and calculate the probability of a sentence

Aim: To generate bigrams and calculate sentence probability using the bigram language model.

A bigram is a sequence of two consecutive tokens. The bigram probability is

\[

P(w_i\mid w_{i-1})=\frac{C(w_{i-1},w_i)}{C(w_{i-1})}

\]

The sentence probability is calculated by multiplying the conditional probabilities of consecutive words.

```
from nltk import word_tokenize, bigrams
from collections import Counter

# A small training corpus.
corpus = [
    "i like natural language processing",
    "i like machine learning",
    "i like language models"
]

# Convert the corpus to lowercase tokens.
tokens = []

for sentence in corpus:
    tokens.extend(word_tokenize(sentence.lower()))

# Add sentence-boundary markers.
tokens = ['<s>'] + tokens + ['</s>']

# Generate bigrams from the token sequence.
bigram_list = list(bigrams(tokens))

# Count individual words and bigram pairs.
unigram_counts = Counter(tokens)
bigram_counts = Counter(bigram_list)

print("Bigram Frequencies:")
for pair, count in bigram_counts.items():
    print(pair, ":", count)

# Calculate the probability of a sentence.
test_sentence = "i like language models"

test_tokens = ['<s>'] + word_tokenize(test_sentence.lower()) + ['</s>']

probability = 1.0

for previous_word, current_word in bigrams(test_tokens):
    # Apply maximum-likelihood estimation.
```

```
    numerator = bigram_counts[(previous_word, current_word)]
    denominator = unigram_counts[previous_word]

    if denominator == 0 or numerator == 0:
        probability = 0.0
        break

    probability *= numerator / denominator

print("\nTest sentence:", test_sentence)
print("Sentence probability:", probability)
```

Sample output

```
Bigram Frequencies:
('<s>', 'i') : 3
('i', 'like') : 3
('like', 'natural') : 1
...

Test sentence: i like language models
Sentence probability: 0.037037037037037035
```

Explanation: The program counts individual words and adjacent word pairs. It then multiplies the conditional probabilities, including the end-of-sentence marker. The sample probability is based on the three training sentences shown above.

Limitation: An unseen bigram makes the unsmoothed probability zero. Larger language models usually apply smoothing techniques to avoid this issue.